Assignment 7: Operations on Processes

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Deadline: two weeks from the date of issue — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation (for a `fork` count: without the process tree) earns partial credit only. Assume every `fork` succeeds, every `printf` is followed by a flush unless the question says otherwise, and all headers are included.
Reading: Silberschatz et al., *Operating System Concepts*, Ch. 3 §3.3, pp. 110–116 [2]; Bovet & Cesati, *Understanding the Linux Kernel*, Ch. 3, pp. 79–130 and Ch. 20, pp. 808–834 [1].

Concepts That Click

7.1 Conceptual

Q1. Every operating system must decide three things when a parent creates a child [2].

(a) Name the three decisions and state, for each, the options available.

(b) Explain how UNIX answers the address-space decision in two calls, and how Windows answers it in one. Where, in each design, does the program that the child will run get its customization (for example, a redirected standard input)?

(c) A shell must run `sort < data.txt`. List, in order, the calls the shell and the child make, and explain why the redirection must happen *between* `fork` and `exec` and not before the `fork` or after the `exec`.

(d) `fork` appears to copy the entire address space only for `exec` to discard it. Explain why this is not expensive on a modern kernel.

Q2. A parent P creates a child C at time $t = 0$. C calls `exit(3)` at $t = 2\,\text{s}$. P sleeps until $t = 10\,\text{s}$ and then calls `waitpid` on C.

(a) State C's condition at $t = 1\,\text{s}$, $t = 5\,\text{s}$, and $t = 11\,\text{s}$. At $t = 5\,\text{s}$, which of C's resources have been released and which kernel record remains, and why?

(b) What value does `WEXITSTATUS` report to P, and what does `waitpid` return?

(c) Now suppose P instead exits at $t = 1\,\text{s}$ without waiting, and C runs until $t = 2\,\text{s}$. Name C's condition at $t = 1.5\,\text{s}$, state who its parent is, and who reaps it.

(d) On a system that enforces cascading termination, what would have happened to C in part (c)?

7.2 Numerical

Q3. For each fragment, give the number of lines printed and the total number of processes (including the original), and draw the process tree. Each fragment is the entire body of `main`, followed by code that reaps all children.

(a) `for (i = 0; i < 5; i++) fork(); printf("p\n");` Also state the number of new children.

(b) `for (i = 0; i < 2; i++) { if (fork()) printf("P\n"); else printf("C\n"); }` Give the counts of `P` and `C` lines separately.

(c) `fork() || fork(); printf("q\n");`

(d) `fork(); fork() && fork(); printf("r\n");`

Q4. Buffering and bookkeeping change the numbers.

(a) With *no* flush anywhere, the fragment `printf("A"); fork(); fork(); printf("B\n");` runs with standard output redirected to a file. How many `A` characters and how many `B` characters does the file contain? Explain, and give the change that makes exactly one `A` appear.

(b) A server forks one child per request at a steady 50 requests per second and never calls `wait`. Each child exits within 10 ms. The system allows at most 32,768 processes (zombies included), and 300 entries are already in use when the server starts. Assuming no other processes are created or reaped, after how many whole seconds does `fork` first fail? How much memory do the accumulated zombies hold in their address spaces at that point?

7.3 Design

Q5. You must create three child processes in two different shapes.

(a) *Fan:* write a loop in C in which one parent creates exactly three children, each child prints its pid and exits immediately, and the parent then reaps all three. Draw the tree and explain what goes wrong if the child does *not* call `exit` inside the loop body (give the resulting process count).

(b) *Chain:* write a loop in which each new process creates exactly one child, so that the result is a chain of four processes (original plus three descendants). Draw the tree, and state which process each process waits for.

Q6. A parent must run three workers and combine their results.

(a) Worker i ($i = 0, 1, 2$) computes a small result and must report it to the parent. Explain why the worker cannot store the result in a global variable for the parent to read, and which mechanism from this lecture it can use instead. State the range of values that mechanism can carry.

(b) Write the C code: the parent forks the three workers, worker i terminates with status $i + 1$, and the parent loops on `wait` until no children remain, summing `WEXITSTATUS` for every child that exited normally. State the printed sum.

Concepts That Click

(c) Modify the design so that the parent prints the results in the order $i = 0, 1, 2$ regardless of which worker finishes first. Which call do you use, and what must the parent record when it forks?

References

[1] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3rd edition, 2006.

[2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

Concepts That Click